PMS Task Overdue Prediction

Feature Engineering & Leakage Mitigation Documentation

Based on clean_and_build_dataset_leak_fixed.ipynb

Reference SQL: analytical_dataset_leak_fixed.sql

Reference Module: build_features_leak_fixed.py

Generated: July 2026

Table of Contents

1. Overview & Architecture

The dataset pipeline builds two clean CSV files from the PMS database: dataset_at_creation_clean.csv and dataset_at_halfway_clean.csv. Each row corresponds to a task that has both start_date and end_date defined (13,895 tasks). The pipeline applies 5 leakage mitigation techniques to ensure only information available at the prediction timepoint is used for each feature.

Prediction Timepoints:
- Creation cutoff = task.created_date (when the task is first assigned)
- Halfway cutoff = GREATEST(created_date, start_date + (end_date - start_date) / 2)
  (the midpoint of the task, capped to not precede creation)

Output Datasets:
- Creation: 13,895 rows, 36 columns (33 features + id + target + target_source)
- Halfway: 13,895 rows, 38 columns (35 features + id + target + target_source)

Tables used: tasks_task, tasks_task_history, tasks_sub_task, tasks_sub_task_history, tasks_major_activity, tasks_major_activity_history, tasks_kpi, tasks_kpi_history, tasks_cross_department_assignments, comments_comment, basedata_position

2. Target Variable: Three-Tier Overdue Calculation

The target (calculated_overdue) is a binary label: 1 if the task was overdue, 0 otherwise. It uses a three-tier approach to handle missing actual_end_date for completed tasks:

Tier 1 - actual_end_date: If the task has an actual_end_date and status = completed, use it directly. Overdue if actual_end_date > end_date.

Tier 2 - first_completed_at: If status = completed but no actual_end_date, use the earliest history_date from tasks_task_history where status = completed.

Tier 3 - updated_date: If status = completed but no actual_end_date and no history completion record, fall back to the task updated_date.

For open tasks (not completed, terminated, or archived): overdue if end_date < fixed cutoff (2026-07-14).

```
-- SQL (analytical_dataset_leak_fixed.sql lines 360-372)
CASE
  WHEN b.status = 'completed' AND b.actual_end_date IS NOT NULL
       AND b.actual_end_date > b.end_date THEN 1
  WHEN b.status = 'completed' AND b.actual_end_date IS NULL
       AND fc.first_completed_at IS NOT NULL
       AND fc.first_completed_at > b.end_date THEN 1
  WHEN b.status = 'completed' AND b.actual_end_date IS NULL
       AND fc.first_completed_at IS NULL
       AND b.updated_date::date > b.end_date THEN 1
  WHEN b.status NOT IN ('completed','terminated','archived')
       AND b.end_date < '2026-07-14'::date THEN 1
  ELSE 0
END AS calculated_overdue
```

Target source breakdown (from notebook output):

actual_end_date:	7,444 (53.6%)	- Tier 1
updated_date:	5,427 (39.1%)	- Tier 3
status_based:	722 (5.2%)	- not overdue, completed on time
history_completion:	231 (1.7%)	- Tier 2
open_task:	71 (0.5%)	- open & past due

Overall overdue rate: 47.81%

NOTE: The target uses full historical information (no cutoff filter). This is correct because the target is the ground-truth label being predicted. Only FEATURES need cutoff-aware logic.

3. Feature Catalog (35 Features)

All 35 features organized by source table and leakage mitigation technique. Features marked "cutoff-aware" use one of 5 techniques to prevent data leakage. Features marked "static" are deterministic from the task's own attributes at creation.

3.1 Derived Temporal Features (9 static)

These features are computed directly from the task's own start_date, end_date, and created_date. They are deterministic and involve no external data, so no cutoff filter is needed.

Feature	Technique	Dtype	Formula
planned_duration	Derived	int	(end_date - start_date).days
creation_to_planned_start	Derived	int	(start_date - created_date).days
created_dow	Derived	int	EXTRACT(DOW FROM created_date)
created_is_weekend	Derived	int	1 if DOW >= 5 else 0
created_is_friday	Derived	int	1 if DOW == 4 else 0
created_month	Derived	int	EXTRACT(MONTH FROM created_date)
created_quarter	Derived	int	EXTRACT(QUARTER FROM created_date)
is_planned	Derived	int	t.is_planned::int
risk_mapping	Derived	int	t.risk_mapping

```
-- SQL from CTE base (analytical_dataset_leak_fixed.sql lines 51-58)
(t.end_date - t.start_date) AS planned_duration,
(t.start_date - t.created_date) AS creation_to_planned_start,
EXTRACT(DOW FROM t.created_date)::int AS created_dow,
CASE WHEN EXTRACT(DOW FROM t.created_date)::int >= 5 THEN 1 ELSE 0 END AS created_is_weekend,
CASE WHEN EXTRACT(DOW FROM t.created_date)::int = 4 THEN 1 ELSE 0 END AS created_is_friday,
EXTRACT(MONTH FROM t.created_date)::int AS created_month,
EXTRACT(QUARTER FROM t.created_date)::int AS created_quarter
```

3.2 Task History Features - REVISIONS (3 cutoff-aware)

Source: tasks_task_history. Original pipeline counted ALL revisions (no cutoff). Leak-fixed uses COUNT(*) FILTER (WHERE history_date <= cutoff) to only count revisions that occurred BEFORE the prediction timepoint.

Key insight: At creation cutoff, num_revisions = 0 for ALL tasks because history_date = created_date is the initial record, not a revision. This is validated in the test suite.

Feature	Technique	Dtype	Construction
num_revisions	FILTER(history)	int	COUNT(*) FILTER (WHERE history_date <= cutoff)
revision_frequency	FILTER(history)	float	num_revisions / planned_duration
revision_recency	FILTER(history)	int	days since last revision <= cutoff

```
-- SQL: CTE revisions (lines 82-92)
SELECT
  th.history_relation_id AS task_id,
  COUNT(*) FILTER (WHERE th.history_date <= b.creation_cutoff)
    AS num_revisions_at_creation,
  COUNT(*) FILTER (WHERE th.history_date <= b.halfway_cutoff)
    AS num_revisions_at_halfway,
  MAX(th.history_date) FILTER (WHERE th.history_date <= b.creation_cutoff)
    AS last_revision_at_creation,
  MAX(th.history_date) FILTER (WHERE th.history_date <= b.halfway_cutoff)
    AS last_revision_at_halfway
FROM tasks_task_history th
JOIN base b ON b.id = th.history_relation_id
GROUP BY th.history_relation_id
```

3.3 Subtask Features (5 cutoff-aware)

Source: tasks_sub_task. Original pipeline counted ALL subtasks regardless of when they were created. Leak-fixed uses COUNT(*) FILTER (WHERE created_date <= cutoff) and also filters completion/overdue status to only subtasks existing at cutoff.

Feature	Technique	Dtype	Construction
num_subtasks	FILTER(created)	int	COUNT(*) FILTER (WHERE created_date <= cutoff)
has_subtasks	FILTER(created)	int	1 if num_subtasks > 0 else 0
subtask_completion_pct	FILTER(created)	float	completed / subtasks at cutoff
subtask_overdue_rate	FILTER(created)	float	overdue / subtasks at cutoff
subtask_completion_pct_at_halfway	FILTER(created)	float	same, at halfway only

```
-- SQL: CTE subtasks (lines 96-108)
SELECT st.task_id,
  COUNT(*) FILTER (WHERE st.created_date <= b.creation_cutoff)
    AS num_subtasks_at_creation,
  COUNT(*) FILTER (WHERE st.status = 'completed'
    AND st.created_date <= b.creation_cutoff)
    AS num_completed_at_creation,
  COUNT(*) FILTER (WHERE st.is_overdue = TRUE
    AND st.created_date <= b.creation_cutoff)
    AS num_overdue_at_creation
FROM tasks_sub_task st
JOIN base b ON b.id = st.task_id
GROUP BY st.task_id
```

3.4 Status Lookup Features (3 cutoff-aware)

Source: tasks_task_history. Original pipeline used the CURRENT task status, which may have changed after the prediction point. Leak-fixed uses DISTINCT ON with history_date <= cutoff ORDER BY history_date DESC to get the latest status AS IT WAS at the prediction timepoint. If no history record exists at the cutoff, falls back to the current task status.

Feature	Technique	Dtype	Construction
status_encoded	DISTINCT ON	int	0=not_started..4=archived at cutoff
approval_status_encoded	DISTINCT ON	int	0=pending..3=rejected at cutoff
lead_approval_status_encoded	DISTINCT ON	int	0=pending..3=rejected at cutoff

```
-- SQL: CTE task_status_at_creation (lines 113-123)
SELECT DISTINCT ON (th.history_relation_id)
  th.history_relation_id AS task_id,
  th.status AS status_at_creation_val
FROM tasks_task_history th
JOIN base b ON b.id = th.history_relation_id
WHERE th.history_date <= b.creation_cutoff
ORDER BY th.history_relation_id, th.history_date DESC
```

3.5 Major Activity Features (3 static)

Source: tasks_major_activity. MA status and approval_status are static properties of the major activity the task belongs to. They are organizational attributes that exist independently of the task and are available at prediction time. No cutoff filter needed. The KPI ID (kpi_id) is used as a foreign key to link KPI features.

Feature	Technique	Dtype	Construction
ma_status_encoded	Static	int	MA status (0=not_started..3=terminated)
ma_approval_status_encoded	Static	int	MA approval (0=pending..3=rejected)
num_ma_revisions	FILTER(history)	int	MA revisions before cutoff

```
-- SQL: CTEs ma_info and ma_revisions (lines 229-247)
-- MA info (static):
SELECT ma.id, ma.status, ma.approval_status, ma.kpi_id
FROM tasks_major_activity ma

-- MA revisions (cutoff-aware via FILTER):
SELECT mah.id, COUNT(*) FILTER (WHERE mah.history_date <= b.creation_cutoff)
FROM tasks_major_activity_history mah
JOIN base b ON b.major_activity_id = mah.id
GROUP BY mah.id
```

3.6 KPI Features (5 cutoff-aware)

Source: tasks_kpi, tasks_kpi_history. Original pipeline used CURRENT KPI status (kpi.is_overdue), which is a significant leak - if the KPI is currently overdue, it strongly implies the task is overdue. Leak-fixed uses DISTINCT ON history lookup with history_date <= cutoff to get KPI state AT the prediction timepoint. KPI revisions use a correlated subquery with history_date <= cutoff.

CRITICAL FIX: The original used current KPI.is_overdue directly. Since KPI status reflects the overall KPI performance, knowing it was overdue at any point requires cutoff filtering. Our DISTINCT ON approach only shows KPI status as it was at creation/halfway.

Feature	Technique	Dtype	Construction
kpi_is_overdue_flag	DISTINCT ON	int	KPI overdue flag at cutoff
kpi_status_ordinal	DISTINCT ON	int	KPI status at cutoff (0..4)
num_kpi_revisions	FILTER(history)	int	KPI revisions before cutoff

```
-- SQL: KPI status at creation (correlated subquery, lines 264-275)
SELECT DISTINCT ON (kh.id)
  kh.id AS kpi_id,
  kh.is_overdue::int AS kpi_is_overdue_flag_at_creation,
  kh.status AS kpi_status_at_creation
FROM tasks_kpi_history kh
JOIN tasks_kpi kpi ON kpi.id = kh.id
JOIN tasks_major_activity ma ON ma.kpi_id = kpi.id
JOIN base b ON b.major_activity_id = ma.id
WHERE kh.history_date <= b.creation_cutoff
ORDER BY kh.id, kh.history_date DESC
```

3.7 Comment Features (1 cutoff-aware)

Source: comments_comment. Original counted ALL comments on a task. Leak-fixed uses COUNT(*) FILTER (WHERE created_date <= cutoff). Content type 24 = task comments.

Feature	Technique	Dtype	Construction
task_comment_count	FILTER(created)	int	comments before cutoff

```
-- SQL: CTE comments_task (lines 292-301)
SELECT cc.object_id AS task_id,
  COUNT(*) FILTER (WHERE cc.created_date <= b.creation_cutoff)
    AS task_comment_count_at_creation
FROM comments_comment cc
JOIN base b ON b.id = cc.object_id
WHERE cc.content_type_id = 24
GROUP BY cc.object_id
```

3.8 Sub-Task Status Churn Feature (1 cutoff-aware)

Source: tasks_sub_task_history. Measures how many distinct statuses a subtask has gone through (a proxy for churn/instability). Original counted ALL history. Leak-fixed uses COUNT(DISTINCT status) FILTER (WHERE history_date <= cutoff), then averages per parent task.

Feature	Technique	Dtype	Construction
avg_sub_status_changes	FILTER(history)	float	avg distinct status changes per subtask at cutoff

```
-- SQL: CTEs sub_hist + task_sub_churn (lines 317-335)
SELECT sth.id AS sub_task_id,
  COUNT(DISTINCT sth.status)
    FILTER (WHERE sth.history_date <= b.creation_cutoff)
    AS num_status_changes_at_creation
FROM tasks_sub_task_history sth
JOIN tasks_sub_task st ON st.id = sth.id
JOIN base b ON b.id = st.task_id
GROUP BY sth.id
```

3.9 Cross-Department Features (2 static)

Source: tasks_task, tasks_cross_department_assignments. These flags are set at task creation and do not change. No cutoff needed.

Feature	Technique	Dtype	Construction
is_cross_dept	Static	int	1 if derived_from_cross_department_assignment_id IS NOT NULL
cross_dept_pair_exists	Static	int	1 if cross-dept assignment record exists

```
-- SQL: CTEs base (is_cross_dept) and cross_dept (lines 49, 339-342)
CASE WHEN t.derived_from_cross_department_assignment_id IS NOT NULL
  THEN 1 ELSE 0 END AS is_cross_dept

SELECT t.id AS task_id, 1 AS cross_dept_pair_exists
FROM tasks_task t
JOIN tasks_cross_department_assignments cda
  ON cda.id = t.derived_from_cross_department_assignment_id
```

3.10 Group Aggregate Features (4 expanding window)

Source: tasks_task, basedata_position. Original pipeline computed aggregates over ALL tasks in the same group (AVG without window). This used the current task's own status AND future tasks. Leak-fixed uses expanding window aggregates with ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING, which computes the running average of ALL PREVIOUS tasks (excluding the current row).

CRITICAL FIX: The original computed these as simple GROUP BY averages, which meant the current task's own status was included (self-leak), and future tasks were included (temporal leak). The expanding window orders by created_date and excludes the current row, solving both issues.

Feature	Technique	Dtype	Construction
dept_past_overdue_rate	Expanding Window	float	Running overdue rate in department before task
dept_avg_revisions	Expanding Window	float	Running avg revisions in department
emp_past_overdue_rate	Expanding Window	float	Running overdue rate for employee
pos_past_overdue_rate	Expanding Window	float	Running overdue rate for position

```
-- SQL: CTE dept_agg_raw + dept_agg (lines 142-175)
SELECT t.id, COALESCE(p.department_id, t.department_id) AS dept_id,
  t.created_date,
  CASE WHEN ... THEN 1 ELSE 0 END AS is_overdue
FROM tasks_task t
LEFT JOIN basedata_position p ON p.id = t.position_id

SELECT id,
  AVG(is_overdue) OVER (
    PARTITION BY dept_id ORDER BY created_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
  ) AS dept_past_overdue_rate
FROM dept_agg_raw
```

3.11 Halfway-Only Features (2 extra, total 35)

These two features exist only in the halfway dataset. They add information available at the task midpoint that was not available at creation.

Feature	Technique	Dtype	Construction
days_since_update	Derived	int	FIXED_CUTOFF - updated_date at halfway
subtask_completion_pct_at_halfway	FILTER(created)	float	subtask completion % at halfway (mirrors creation feature)

The remaining 14 features (status_encoded, approval_status_encoded, num_revisions, etc.) share the same name across creation and halfway but contain different values because they are recomputed at the halfway cutoff.

4. Leakage Mitigation Techniques

Five techniques are used across the pipeline. Each technique addresses a specific pattern of data leakage:

Technique	Applied To	How It Prevents Leakage
FILTER (history_date)	Revisions, MA revs, KPI revs, sub-task chur	Count/aggregate only history records with date <= cutoff
FILTER (created_date)	Subtasks, Comments	Count only records created on or before the cutoff
DISTINCT ON history lookup	Status, approval, KPI status	Latest history record at cutoff; fallback to current value
Expanding window	dept/emp/pos aggregates	Running avg of previous tasks only, excluding current row
Dropped	8 challenge features, position_id_encoded	Junction tables lack date columns; encoding is fold-safe only

4.1 Technique 1: FILTER (history_date <= cutoff)

Applies to: revisions (num_revisions), MA revisions (num_ma_revisions), KPI revisions (num_kpi_revisions), sub-task status churn (avg_sub_status_changes).

How it works: The SQL COUNT/MAX/SUM aggregation uses a FILTER clause that restricts which rows are included based on whether their history_date is at or before the cutoff.

Original leak:COUNT(*) without filter counted ALL history rows, including revisions made after the prediction point. This means the model would "know" about future changes.

Verification: At creation cutoff, num_revisions = 0 for all 13,895 tasks because no history exists before creation. At halfway, only revisions up to the midpoint are counted.

4.2 Technique 2: FILTER (created_date <= cutoff)

Applies to: subtasks (num_subtasks, completion, overdue), comments (task_comment_count).

How it works: Same FILTER pattern, but filtering on created_date instead of history_date. For subtasks, also applied to completion/overdue status to avoid knowing future outcomes.

Original leak:COUNT(*) without filter counted ALL subtasks, including ones created after the prediction point. The subtask_completion_pct used current st.status, which reveals whether the subtask was eventually completed - future information at prediction time.

Verification: At creation cutoff, num_subtasks = 0 for all tasks (subtasks are created after the parent task). At halfway, only subtasks created up to the midpoint are included.

4.3 Technique 3: DISTINCT ON History Lookup

Applies to: status_encoded, approval_status_encoded, lead_approval_status_encoded, kpi_is_overdue_flag, kpi_status_ordinal.

How it works: Uses PostgreSQL DISTINCT ON combined with ORDER BY history_date DESC to get the most recent history record AT or before the cutoff. This gives the "snapshot" of the state at prediction time. If no history record exists at cutoff (e.g., at creation for task status), it falls back to the current value from the main table.

Original leak: The original pipeline read CURRENT status (t.status), which could be "completed" even at creation time if the task was completed quickly. This is future information. For KPI, current kpi.is_overdue directly reveals if the KPI is overdue,
which is almost collinear with the task being overdue.

Key insight: At creation cutoff, the DISTINCT ON often finds the initial history record (history_date = created_date) with the task's initial status. Approximately 99% of tasks have a non-not_started status at creation because tasks are often created with "ongoing" status.

4.4 Technique 4: Expanding Window Aggregates

Applies to: dept_past_overdue_rate, dept_avg_revisions, emp_past_overdue_rate, pos_past_overdue_rate.

How it works: Instead of a GROUP BY aggregate (which includes the current task and all tasks), uses a window function with ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING. This creates a running aggregate that only considers tasks created BEFORE the current task.

Original leak: The original used GROUP BY with AVG over ALL tasks in the department, including the current task itself (self-leak) and tasks created after it (temporal leak).

Remaining concern (noted in handoff): These features are pre-computed in the CSV. For cross-validation, they need fold-safe recomputation because the pre-computed values still use all data. The model_training.ipynb handles this by creating fold-safe variants that exclude these 4 features and quantifying the leakage gap.

```
-- Expanding window detail: only previous tasks, ordered by created_date
AVG(is_overdue) OVER (
  PARTITION BY dept_id
  ORDER BY created_date
  ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
) AS dept_past_overdue_rate

-- The "1 PRECEDING" excludes the current row (self-leakage prevention)
-- The "UNBOUNDED PRECEDING" includes all prior tasks (temporal correctness)
```

4.5 Technique 5: Dropped Features

Applies to: 8 challenge features + position_id_encoded.

Challenge features dropped:
- num_challenges, has_challenges (from tasks_task_challenge_groups)
- has_subtask_challenge, num_subtask_challenges (from tasks_sub_task_challenge_groups)
- has_kpi_challenge, num_kpi_challenges (from tasks_kpis_challegne_groups)
- has_kpi_potential_challenge, num_kpi_potential_challenges (from tasks_kpis_potential_challenge_groups)

Why dropped: These junction tables lack date columns, making it impossible to determine whether a challenge was recorded before or after the prediction point. Without a date, we cannot apply any cutoff filter, so these features would leak future information.

position_id_encoded dropped:
- This is a target-encoded feature (position_id -> mean overdue rate). Pre-computing it on the full dataset leaks target information. It should only be computed within CV folds.
- The encoding is performed inside the model_training notebook as part of fold-safe CV.

5. Features Dropped vs Original Pipeline

Compared to the original dataset_at_halfway.csv (51 columns), the clean version (38 columns) has 14 fewer columns:

Feature	Status	Reason
num_challenges	DROPPED	No date column in junction table
has_challenges	DROPPED	No date column
has_subtask_challenge	DROPPED	No date column
num_subtask_challenges	DROPPED	No date column
has_kpi_challenge	DROPPED	No date column
num_kpi_challenges	DROPPED	No date column
has_kpi_potential_challenge	DROPPED	No date column
num_kpi_potential_challenges	DROPPED	No date column
position_id_encoded	DROPPED	Fold-safe only
kpi_comment_count	DROPPED	Removed from dropped list
ma_comment_count	DROPPED	Removed from dropped list
wl_high	DROPPED	Weight level features
wl_mid	DROPPED	Weight level features
wl_low	DROPPED	Weight level features

1 column added: target_source (metadata tracking which tier was used for target).
37 columns are common between original and clean, but all dynamic features are rebuilt with cutoff-aware logic.

6. Dual-Cutoff Architecture

Each feature that depends on time is computed TWICE - once at the creation cutoff and once at the halfway cutoff. This enables comparing model performance at two distinct prediction points:

Creation (33 features): What we know when the task is first assigned.
Halfway (35 features): What we know at the task midpoint (more information).

Features that share the same name but differ between datasets:

Feature	Technique	Creation vs Halfway
status_encoded	DISTINCT ON	Different snapshot at each cutoff
approval_status_encoded	DISTINCT ON	Different snapshot at each cutoff
lead_approval_status_encoded	DISTINCT ON	Different snapshot at each cutoff
num_ma_revisions	FILTER(history)	More revisions by halfway
kpi_is_overdue_flag	DISTINCT ON	Updated KPI state at halfway
kpi_status_ordinal	DISTINCT ON	Updated KPI status at halfway
num_kpi_revisions	FILTER(history)	More KPI revisions by halfway
num_revisions	FILTER(history)	Task revisions by halfway
revision_frequency	FILTER(history)	Updated at halfway
revision_recency	FILTER(history)	Days since last revision at halfway
num_subtasks	FILTER(created)	More subtasks by halfway
has_subtasks	FILTER(created)	Flag updated at halfway
task_comment_count	FILTER(created)	More comments by halfway
avg_sub_status_changes	FILTER(history)	More churn by halfway

Halfway-only features: days_since_update, subtask_completion_pct_at_halfway.
Static features (identical at both cutoffs): planned_duration, creation_to_planned_start, created_dow, created_is_weekend, created_is_friday, created_month, created_quarter, is_planned, risk_mapping, is_cross_dept, cross_dept_pair_exists, dept_past_overdue_rate, dept_avg_revisions, emp_past_overdue_rate, pos_past_overdue_rate, ma_status_encoded, ma_approval_status_encoded.

7. Validation Checks

The test suite (tests/v1/test_data_validation_leak_fixed.py) validates:

Creation Dataset Tests:
- 13,895 rows, 36 columns (33 features)
- Column order matches expected schema
- No duplicate IDs, no nulls
- Target rate 47.8% (validated range 46-49%)
- Target is binary (0 or 1)
- All 9 dropped features absent
- status_encoded values in valid range 0-4

Halfway Dataset Tests:
- 13,895 rows, 38 columns (35 features)
- Contains all creation columns + 2 extra
- Extra columns match expected: subtask_completion_pct_at_halfway, days_since_update
- No duplicate IDs, no nulls
- Same IDs and target as creation

Leakage Validation Tests:
- num_revisions at creation = 0 for ALL tasks (pass)
- num_revisions non-decreasing (pass)
- subtasks non-decreasing (pass)
- Same target across datasets (pass)
- No challenge features in either dataset (pass)
- No position_id_encoded in either dataset (pass)

The validation in the notebook itself also checks:
- num_revisions at creation = 0 (expect 0)
- Revisions at halfway exist: 3,250 tasks have revisions
- Subtasks at creation = 0, at halfway = 91 tasks have subtasks
- Comments at creation = 0, at halfway = 3 tasks have comments

All checks pass.